

Manifiesto de Arquitectura Evolutiva y Gestión de Activos Digitales (ACIDE)

1. Introducción y Visión Estratégica

1.1. Propósito del Manifiesto: De “Código de Proyecto” a “Activos de Negocio”

En **ACIDE**, rechazamos el modelo de desarrollo basado en silos, donde cada sistema se construye desde cero como una pieza única y aislada. Este enfoque genera redundancia, errores repetitivos y una pérdida masiva de conocimiento técnico y funcional.

Este manifiesto establece un cambio de paradigma: **dejamos de construir proyectos aislados para construir un ecosistema de activos de negocio reutilizables.**

- Pasamos de programar “funciones para el Cliente X” a perfeccionar “Módulos de Proceso” (ej. Gestión de Inventarios, Procesamiento de Pagos, Registro de Entidades).
- El software deja de ser un gasto operativo y se convierte en un **activo intelectual** que incrementa su valor y robustez con cada nueva implementación.

1.2. Objetivos de Negocio: Rentabilidad y Escalabilidad

La arquitectura no es una preferencia estética; es una decisión financiera y estratégica. Implementamos este estándar para alcanzar tres pilares críticos:

1. **Aceleración del Time-to-Market:** Al iniciar un sistema con módulos base extraídos de nuestro “Garaje”, eliminamos la fase de diseño estructural desde cero, permitiendo que el equipo se enfoque exclusivamente en las reglas particulares del nuevo negocio.
2. **Fiabilidad Heredada:** Un módulo que ha sido implementado y probado en múltiples entornos es inherentemente más robusto. Reducimos drásticamente la incertidumbre y los fallos críticos en producción.
3. **Independencia y Estabilidad (Snapshot & Fork):** Protegemos la integridad de los sistemas en producción. Al entregar una “fotografía” funcional del módulo al proyecto, evitamos el riesgo de que una actualización global rompa sistemas de clientes antiguos que no requieren cambios.

1.3. Audiencia y Alcance: Nuestra Constitución Técnica

Este documento es la **Constitución Técnica de ACIDE** y su cumplimiento es obligatorio para garantizar la viabilidad a largo plazo de la empresa.

- **Arquitectos y Líderes Técnicos:** Son los guardianes del Manifiesto. Su éxito se mide en la capacidad de “extraer” valor de los proyectos locales

hacia el repositorio central.

- **Desarrolladores Backend:** Son los ejecutores. Deben aplicar las capas de abstracción y reglas de dependencia en cada línea de código para asegurar que el módulo sea “desprendible” en el futuro.
- **Product Owners:** Deben entender que el respeto a esta estructura garantiza que el producto pueda escalar y pivotar sin necesidad de una reescritura total.

Alcance: Este manifiesto rige para todo nuevo desarrollo de backend y para cualquier refactorización mayor que aspire a formar parte del catálogo de activos de ACIDE.

2. El Ecosistema de Software (Repositorios)

En **ACIDE**, el software se organiza para separar las herramientas tecnológicas de las capacidades de negocio. Esta división permite que seamos rápidos al iniciar y flexibles al crecer, sin perder nunca la calidad estructural.

2.1. El Repositorio de Módulos Base (El Garaje)

Es nuestra biblioteca de **plantillas funcionales**. No es un código estático; es un estándar de negocio que perfeccionamos continuamente.

- **Función:** Actúa como el “molde” inicial para procesos de negocio (ej: inventarios, pagos, usuarios).
- **Evolución:** Cuando descubrimos una mejor forma de resolver un proceso general, actualizamos el molde para que los **futuros proyectos** nazcan con esa mejora.
- **Uso:** Estos módulos se **copian** al nuevo proyecto para que sirvan de cimiento. No se instalan como dependencias externas que bloquean el cambio.

2.2. El Proyecto Local (El Código del Cliente)

Es el repositorio donde se construye la solución específica (ej: Sistema para Empresa X).

- **Autonomía y Propiedad:** Una vez que un módulo del “Garaje” se copia a la carpeta **src/**, el código pertenece al proyecto. El equipo puede adaptarlo a las necesidades particulares del cliente.
- **Límites de la Autonomía:** La libertad de modificar el código **no es una licencia para el desorden**. Aunque el código sea local, debe respetar obligatoriamente la **Estructura de Capas (Clean Architecture)** y los **Patrones de Diseño** definidos en este manifiesto.
- **Integridad:** Un módulo desordenado en el Proyecto A es un módulo que no podrá ser “extraído” para mejorar el Garaje. Mantener la estructura es lo que permite que el conocimiento fluya de regreso al origen.

2.3. Las Librerías Técnicas (libs/)

Son herramientas puramente tecnológicas que proporcionan servicios de infraestructura.

- **Responsabilidad:** Proveer “enchufes” técnicos (ej: `lib-mailer`, `lib-storage`, `lib-auth`).
- **Regla de Oro: Cero procesos de negocio.** Las librerías son piezas intercambiables. El negocio en `src/` no debe depender de cómo funciona la librería por dentro, sino de la interfaz que esta provee.

3. Modelo de Evolución: Snapshot & Fork

Este modelo define cómo el software nace, se adapta y, finalmente, cómo el conocimiento regresa al origen para beneficiar a futuros proyectos.

3.1. Fase de Inicio: El “Snapshot” (La Foto)

Cuando nace un nuevo proyecto, el arquitecto o líder técnico selecciona los módulos necesarios del **Garaje**.

- **Acción:** Se realiza una copia física de la versión actual del módulo (ej. `modulo-pagos`) y se pega directamente en la carpeta `src/` del proyecto destino.
- **Resultado:** El proyecto cuenta con un cimiento sólido y probado, reduciendo el tiempo de configuración inicial en un **70%**.

3.2. Fase de Adaptación: El “Fork” (La Bifurcación)

Una vez inyectado en el proyecto local, el código entra en su fase de vida independiente.

- **Personalización:** El equipo ajusta el módulo a las reglas específicas del cliente.
- **Aislamiento de Riesgo:** Cualquier cambio “rompedor” (*breaking change*) realizado en este proyecto **no afecta** a otros proyectos ni al Garaje original.
- **Propiedad:** El equipo del proyecto es responsable de mantener la calidad de este código, respetando siempre las capas y patrones definidos en el Punto 2.

3.3. Protocolo de Auditoría y Consenso Técnico

Para que una mejora o corrección local sea “ascendida” al **Garaje**, el proponente debe someter su trabajo a un proceso de auditoría formal. No se aceptan propuestas verbales; se auditan realidades.

A. El Paquete de Entrega (Requisitos de Propuesta)

El desarrollador debe presentar un ticket o documento breve que contenga:

1. **Justificación Escrita:** Explicación del problema detectado o la oportunidad de mejora y por qué beneficia a futuros sistemas (el “Por qué”).
2. **Acceso al Código:** El código debe estar limpio, siguiendo las capas de Clean Architecture y listo para ser revisado en un entorno de pruebas o rama específica.
3. **Impacto Previsto:** Declarar si el cambio es compatible con lo anterior o si es un *Breaking Change* que requiere especial atención.

B. Ventana de Auditoría (48h a 1 Semana)

Una vez entregado el paquete, se abre una ventana de revisión profunda. El tiempo dependerá de la complejidad del cambio (definido por el Arquitecto):

- **Cambios Menores/Correcciones:** 48 horas de revisión.
- **Mejoras Estructurales/Nuevos Módulos:** Hasta 1 semana de revisión.
- **Dinámica:** Cada miembro del equipo debe revisar el código a detalle. No es una lectura superficial; es una validación de que el cambio no rompa el **Manifiesto**.

C. Aprobación por Consenso Documentado

- **Votación Informada:** La aprobación solo es válida si el revisor confirma que ha leído la justificación y ha auditado el código.
- **Silencio Administrativo:** Si tras cumplirse el plazo (48h o 1 semana) un miembro no ha emitido comentarios, se asume su conformidad con la decisión de la mayoría y del Arquitecto.

D. Registro de Disidencia Fundamentada (Voto en Contra)

Si un miembro del equipo **no está de acuerdo** con la integración de una mejora al Garaje, tiene el derecho y la obligación de documentar su postura:

1. **Justificación Técnica:** Debe escribir una breve nota explicando por qué considera que el cambio es un riesgo (ej: “rompe la independencia de capas”, “agrega complejidad innecesaria”, “es demasiado específico para un solo cliente”).
2. **Trazabilidad del Riesgo:** Esta objeción quedará grabada en el historial de la versión.
3. **Valor del Desacuerdo:** Si en el futuro la mejora integrada causa errores o deuda técnica, el registro de disidencia servirá para auditar el proceso de decisión y ajustar el criterio del Arquitecto y del equipo.

3.4. Registro de Cambios y Documentación (El Libro de Bitácora)

El registro de cambios es la memoria viva de **ACIDE**. Se rige por la inmediatez y la transparencia:

1. **Plazo de Registro (Límite 24h):** Tras la aprobación, el proponente tiene 24 horas para formalizar el asiento en la Bitácora. **Sin registro, no hay Merge al Garaje.**
2. **Contenido del Registro:**
 - **Resumen Técnico:** Qué cambió y qué archivos se afectaron.
 - **Origen:** Proyecto de donde nace la mejora.
 - **Panel de Auditores:** Lista de miembros que revisaron y aprobaron.
 - **Manual de Uso:** Breve explicación de cómo implementar esta mejora en el siguiente proyecto.

3.5. Gestión de Riesgos: El Peligro de la Divergencia Salvaje

El modelo *Snapshot & Fork* nos da agilidad, pero conlleva un riesgo crítico que todo el equipo debe monitorear: **La desvinculación total del estándar.**

- **El Riesgo:** Si el Proyecto A, el Proyecto B y el Proyecto C modifican el mismo módulo (ej. *matricula*) de formas radicalmente distintas y sin seguir los patrones de capas, el “Garaje” perderá su valor. El equipo ya no sabrá qué versión usar como base para el siguiente cliente.
- **La Mitigación:** La autonomía local **no es una licencia para el caos.**
 1. Aunque el código sea local, la estructura de capas (*domain, application, infra*) es innegociable.
 2. Si un cambio en el proyecto local es tan específico que rompe la lógica general del sector, debe estar debidamente comentado y aislado.
 3. Es responsabilidad de cada miembro del equipo evitar que los proyectos se conviertan en “islas tecnológicas” imposibles de entender por otros compañeros.

Llegamos al punto donde el **Manifiesto de ACIDE** se convierte en código real. El **Punto 4** es el manual de construcción: si un módulo no se ve así, no entra al “Garaje” y no se aprueba en el Proyecto Local.

Aquí definimos la anatomía exacta de nuestro estándar para que sea “desenchufable” y reutilizable.

4. Estándares de Estructura de Proyecto

Para que un módulo nacido en un proyecto pueda ser extraído al “Garaje” y reutilizado en futuros sistemas, debe seguir una estructura de **Capas de Respon-**

sabibilidad Única. No negociamos la posición de los archivos; la uniformidad es lo que nos da velocidad.

4.1. Anatomía del Proyecto (Raíz)

Cada sistema en ACIDE debe presentar esta jerarquía clara:

Plaintext

```
/
  libs/                # HERRAMIENTAS TÉCNICAS (Compartidas/Inyectadas)
  src/                 # CAPACIDADES DE NEGOCIO (El Producto)
    [nombre-modulo]/  # Ej: facturacion, inventario, matricula
      api/            # Puerta de entrada
      application/    # Orquestación del proceso
      domain/         # El Corazón (Reglas puras)
      infra/          # Implementación técnica
    <shared>/         # Tipos y constantes SOLO de este proyecto
  main.ts              # Punto de arranque
  .acide-version       # Registro de qué versión del Garaje se usó
```

4.2. Definición de las Capas Internas (src/[modulo]/)

A. Capa de Domain (El Activo Intelectual)

Es la capa más interna y protegida. Contiene la lógica que define al sector de negocio.

- **Contenido:** Entidades, Objetos de Valor (Value Objects) e **Interfaces (Contratos)**.
- **Regla Inquebrantable: Dependencia Cero.** No puede importar nada de *infra*, *api* ni de *libs*. Si el dominio necesita guardar un dato, define una interfaz (ej. *IUserRepository*); la implementación real ocurre fuera.

B. Capa de Application (Los Casos de Uso)

Es el director de orquesta. No decide las reglas, solo ejecuta los pasos.

- **Contenido:** Servicios de aplicación, Comandos, Consultas y DTOs internos.
- **Función:** “Si el usuario existe (Domain), entonces guarda el registro (Infra) y envía un correo (Lib)”.
- **Dependencia:** Solo conoce al Domain.

C. Capa de Infra (Los Adaptadores)

Es donde el software toca la realidad técnica.

- **Contenido:** Repositorios de base de datos, implementaciones de contratos del dominio, clientes de APIs externas.
- **Función:** Traducir los deseos del **Domain** a sentencias SQL, llamadas REST o archivos en disco.
- **Dependencia:** Conoce al **Domain** (para implementar sus interfaces) y puede usar las **libs**.

D. Capa de API / Presentation (La Entrega)

Es el punto de contacto con el mundo exterior.

- **Contenido:** Controladores, Validadores de entrada (Request), Transformadores de salida (Resources).
- **Función:** Recibir la petición, validar el formato y pasarle el trabajo a **Application**.
- **Dependencia:** Conoce a **Application** y **Domain**.

4.3. Estándares de Comunicación entre Módulos

Para evitar que **src/facturacion** se “pegue” a **src/inventario**:

1. **Prohibido el Cross-Import:** Un módulo no puede importar archivos internos de otro.
2. **Comunicación por Interfaz o Eventos:** Si **Facturación** necesita saber algo de **Inventario**, debe hacerlo mediante un evento compartido o una interfaz definida en un espacio común, nunca accediendo a la **infra** del otro módulo.

El “Filtro de Auditoría”

Cualquier propuesta al “Garaje” será rechazada si:

- Un archivo en **domain** tiene un **import** de una librería externa (ej. un ORM).
- La lógica de negocio (ej. cómo se calcula un descuento) está escrita en el **Controller** (API).
- El módulo no puede funcionar si le quitas la base de datos actual (Falta de abstracción en **Infra**).

5. Reglas de Interacción y Dependencia (Invariantes)

Para garantizar que nuestros activos sean realmente “desenchufables” y que el sistema no se convierta en una red de dependencias imposible de mantener, establecemos las siguientes leyes inquebrantables.

5.1. La Ley del Dominio Aislado (Regla de Oro)

El **Domain** es el corazón del negocio y debe permanecer puro.

- **Prohibición:** Ningún archivo dentro de la carpeta **domain/** puede importar nada que provenga de **infra/**, **api/**, o de las **libs/** técnicas.
- **Razón:** Si el dominio depende de una librería de base de datos o de un servicio de correos, ese módulo ya no es reutilizable; está “casado” con una tecnología específica. El dominio solo conoce **Interfaces**.

5.2. Aislamiento de Módulos (Cero Acoplamiento)

Los módulos en **src/** deben ser compartimentos estancos.

- **Prohibición:** Está estrictamente prohibido que un módulo (ej. **src/pagos/**) importe archivos internos (entities, services, repositories) de otro módulo (ej. **src/inventario/**).
- **Cómo interactuar:** Si el Módulo A necesita información del Módulo B:
 1. Usa un **Evento de Dominio** (comunicación asíncrona).
 2. Usa una **Interfaz compartida** definida en un espacio común, pero nunca accede a la implementación privada del otro.

5.3. El Muro de las Librerías (**libs/**)

Las librerías son herramientas, no son el negocio.

- **Prohibición:** No se permite lógica de negocio de ACIDE dentro de la carpeta **libs/**.
- **Criterio de Validación:** Si una librería en **libs/** contiene palabras como “Factura”, “Alumno”, “Matrícula” o “Producto”, está mal ubicada. Esas son reglas de negocio y deben vivir en un módulo en **src/**. Las **libs/** solo conocen términos técnicos (bytes, tokens, sockets, emails).

5.4. Inversión de Dependencia Obligatoria

No programamos contra implementaciones, programamos contra **contratos**.

- **Regla:** La capa de **Application** solicita una acción (ej. “Guardar Usuario”) a través de una Interfaz definida en el **Domain**. La implementación real (el código que toca la base de datos) vive en **Infra**.
- **Consecuencia:** Esto nos permite cambiar de base de datos o de proveedor de servicios sin tocar una sola línea de la lógica de negocio.

5.5. Propiedad del Dato

Cada módulo es dueño de su propia persistencia.

- **Regla:** Un módulo no debe consultar directamente las tablas o esquemas de base de datos de otro módulo. Si **Facturación** necesita saber el stock

de un **Producto**, debe pedírselo al Módulo de **Inventario** mediante su API interna o un servicio de aplicación, nunca entrando a su base de datos.

5.6. Comunicación Inter-Modular (Estrategias de Desacoplamiento)

Para mantener la promesa de que un módulo sea “desenchufable”, prohibimos las llamadas directas entre clases de distintos módulos. Usaremos uno de estos tres caminos:

Opción A: Eventos de Dominio (La preferida para el Garaje)

Es la forma más limpia. El Módulo A no sabe que el Módulo B existe; solo anuncia que algo sucedió.

- **Ejemplo:** El módulo de **Ventas** lanza un evento: **OrdenCreada**.
- **Implementación:** Si en tu proyecto local **SÍ** necesitas inventario, el módulo de **Inventario** se suscribe a ese evento y descuenta stock.
- **Ventaja:** Si en otro proyecto **NO** necesitas inventario, simplemente no copias la carpeta de **Inventario**. El módulo de **Ventas** seguirá lanzando su evento al vacío y nada se romperá. **No tienes que limpiar código.**

Opción B: Interfaces Compartidas (Contratos en Shared)

Si necesitas una respuesta inmediata (síncrona), usamos contratos.

- **Estructura:** Creas una interfaz en una carpeta **src/shared/contracts** (ej: **IPaymentProvider**).
- **Implementación:** El módulo de **Facturacion** usa esa interfaz.
- **El Truco de ACIDE:** En el “Garaje”, esa interfaz no tiene implementación. Cuando llevas el módulo al proyecto, tú decides quién implementa esa interfaz (un módulo real de **Pagos** o un simple *Mock* que siempre devuelve “OK”).
- **Ventaja:** El módulo de **Facturacion** es feliz mientras alguien (cualquiera) respeta el contrato.

Opción C: Orquestación en la Capa de Aplicación

Nunca permitas que un **Domain** llame a otro **Domain**.

- **Regla:** Si un proceso requiere de dos módulos, la lógica de unión debe estar en un **Service de Aplicación** fuera de los módulos o en el punto de entrada (API).
- **Acción:** El controlador recibe la petición y llama primero al **ModuloA.ejecutar()** y luego al **ModuloB.ejecutar()**.

- **Ventaja:** Los módulos siguen siendo piezas de LEGO independientes.

5.7. Prohibición de Herencia Inter-Modular

Para garantizar que cada activo del Garaje sea “desenchufable”, establecemos reglas estrictas sobre cómo compartir lógica sin crear dependencias rígidas.

A. Prohibición de Herencia Transversal

- **Regla:** Está estrictamente prohibido que una clase de un módulo (ej. `FacturaEntity` en `src/pagos`) herede de una clase de otro módulo (ej. `DocumentoBase` en `src/shared` o `src/core`).
- **Razón:** La herencia es el acoplamiento más fuerte que existe. Si heredas, heredas también los errores, las dependencias y la deuda técnica del padre. Rompes la posibilidad de llevarte el hijo solo.

B. La Solución: Composición (Inyección de Comportamiento)

Si el **Módulo B** necesita funcionalidades que ya existen en el **Módulo A**, no “es un” Módulo A, sino que “usa” una capacidad.

1. **Uso de Interfaces:** El Módulo B define una interfaz de lo que necesita (ej. `ICalculadorDeImpuestos`).
2. **Inyección en Tiempo de Ejecución:** En el proyecto local, tú decides si le pasas la implementación del Módulo A o una versión simplificada (Mock) si no quieres traer el módulo completo.

Matriz de Importaciones Prohibidas (Resumen)

Capa Origen	Capa Destino	¿Permitido?	Razón
Domain	Infra / Libs	NO	El núcleo no debe conocer la tecnología.
Application	Infra	NO	El flujo no debe depender de la implementación.
Modulo A	Modulo B	NO	Evita el efecto dominó (si borras B, muere A).
Libs	Cualquier Modulo	NO	Las herramientas no deben conocer el negocio.

6. Calidad y Gobernanza

En **ACIDE**, la calidad no es un “extra”, es el requisito mínimo para la existencia de un módulo. Este capítulo define los criterios de aceptación y el proceso de vigilancia arquitectónica.

6.1. Definition of Done (DoD) para un Módulo

Para que un módulo sea aceptado en el Proyecto Local y, eventualmente, sea candidato al **Garaje**, debe cumplir con los siguientes puntos:

1. **Aislamiento de Capas:** El **Domain** no tiene dependencias externas (Cero imports de `infra` o `libs`).
2. **Contratos Definidos:** Todas las interacciones con bases de datos o servicios externos se realizan mediante interfaces en el **Domain**.
3. **Independencia de Negocio:** El módulo puede funcionar (lógica y pruebas) sin necesidad de que otros módulos de negocio estén presentes.
4. **Documentación de Contrato:** Debe incluir un archivo `README.md` local que explique:
 - ¿Qué problema de negocio resuelve?
 - ¿Qué eventos dispara o qué interfaces requiere para funcionar?
5. **Pruebas de Lógica:** Las reglas críticas del **Domain** deben tener pruebas unitarias que validen su comportamiento agnóstico a la base de datos.

6.2. El Proceso de Revisión de Código (Code Review)

El Líder y el equipo actúan como auditores de la “Constitución Técnica”.

- **Filtro de Arquitectura:** Antes de revisar la lógica funcional, se revisa la estructura. Si hay un `import` prohibido (según el Punto 5), el PR se rechaza automáticamente sin revisar el resto.
- **Criterio de “Extraibilidad”:** Durante la revisión, el equipo debe preguntarse: *¿Podríamos copiar esta carpeta ahora mismo a un proyecto de otro sector y funcionaría tras cambiar los adaptadores de Infra?* Si la respuesta es NO, el código requiere refactorización.

6.3. El Consejo de Arquitectura (La Democracia Técnica)

Para evitar cuellos de botella y fomentar el aprendizaje, la gobernanza es compartida:

1. **Reuniones de Corte:** Al finalizar un ciclo de proyectos, el equipo se reúne para auditar las mejoras propuestas al Garaje.
2. **Votación y Registro:** Se aplica la regla de las **48 horas / 1 semana** de auditoría y el **Silencio Administrativo**.
3. **Asignación de “Guardianes”:** Cada módulo del Garaje puede tener un “Guardián” (desarrollador que mejor conoce ese proceso), quien liderará la documentación de sus *Release Notes*.

6.4. Manejo de Excepciones y Deuda Técnica

Entendemos que hay urgencias comerciales extremas.

- **Excepción Temporal:** Si por una urgencia crítica se debe romper una regla del Manifiesto, debe ser aprobada por el Arquitecto y **registrada como Deuda Técnica** con una fecha de resolución obligatoria en la siguiente “Ventana de Integración”.
- **Prohibición de “Parche Eterno”:** Ninguna excepción puede ser subida al **Garaje**. El Garaje solo acepta código que cumple al 100% con el estándar.

7. Política de Uso de Inteligencia Artificial (IA)

En **ACIDE**, abrazamos la innovación, pero priorizamos la **Integridad Arquitectónica**. La IA es una herramienta de apoyo, no un sustituto del criterio del ingeniero.

7.1. Uso con Moderación y Criterio

- **Prohibición de “Caja Negra”:** No se permite subir al Garaje ni al Proyecto Local código generado por IA que el desarrollador no sea capaz de explicar, refactorizar o mantener manualmente.
- **Responsabilidad de Capas:** El desarrollador es el único responsable de asegurar que el código sugerido por la IA respete las **Invariantes (Punto 5)**. La IA a menudo intenta mezclar capas para “ahorrar tiempo”; es deber del dev separar el **Domain** de la **Infra**.

7.2. Validación de Alucinaciones Técnicas

- Toda lógica de negocio crítica (especialmente en el **Domain**) generada o sugerida por IA debe ser validada mediante **Pruebas Unitarias** escritas o supervisadas por el humano.
- No aceptamos sugerencias de librerías externas obsoletas o inexistentes que la IA pueda inventar.

7.3. El Filtro del Auditor

Si durante una revisión de código (PR) el desarrollador no puede justificar una decisión técnica de su código porque “así lo sugirió la IA”, el cambio será rechazado inmediatamente. **La IA propone, el Ingeniero de ACIDE dispone.**

8. Glosario de Términos

Para garantizar una comunicación técnica sin ambigüedades, definimos los siguientes términos bajo el contexto del ecosistema de **ACIDE**:

- **Activo de Negocio:** Código que encapsula un proceso de sector (ej. Matrícula, Inventario) y que tiene valor comercial por sí mismo, independientemente del proyecto donde nació.
- **Adaptador (Adapter):** Implementación técnica que permite a un módulo hablar con el mundo exterior (Base de Datos, APIs externas). Vive en la capa de **Infra**.
- **Capa de Aplicación (Application Layer):** Capa que orquesta el flujo de un proceso. Sabe *qué* pasos seguir, pero no conoce las reglas del negocio ni los detalles técnicos.
- **Capa de Dominio (Domain Layer):** El núcleo del software. Contiene las reglas de negocio puras, entidades y contratos. Es la única capa que no tiene dependencias externas.
- **Capa de Infraestructura (Infra Layer):** Capa que contiene las herramientas y detalles de implementación (SQL, ORMs, Clientes HTTP).
- **Contrato (Interface):** Definición de “qué” debe hacer una función o servicio, sin decir “cómo” lo hace. Permite que el **Domain** pida acciones sin acoplarse a la tecnología.
- **Divergencia Salvaje:** Riesgo arquitectónico donde un módulo local se aleja tanto del estándar original que se vuelve imposible de entender o de reintegrar al Garaje.
- **Entidad (Entity):** Objeto de negocio que tiene una identidad única y un ciclo de vida (ej. un **Alumno** o una **Factura**).
- **Garaje (Repositorio Original):** El repositorio central de **ACIDE** que almacena las versiones más limpias y potentes de nuestros módulos base.
- **Librería (Lib):** Herramienta técnica agnóstica al negocio (ej. un servicio para subir archivos a S3). No debe contener lógica de procesos de **ACIDE**.
- **Módulo:** Unidad funcional independiente que reside en **src/** y que resuelve un problema de negocio específico de principio a fin.
- **Proyecto Local:** El repositorio específico de un cliente donde los módulos del Garaje se inyectan y cobran vida.
- **Release Notes (Bitácora):** Documentación obligatoria que explica los cambios, autores y justificaciones de una actualización en el Garaje.
- **Snapshot & Fork:** Nuestra estrategia de “Copiar y Bifurcar”. Tomamos una foto del Garaje (Snapshot) y la hacemos evolucionar de forma independiente en el proyecto (Fork).

Acta de Aprobación y Compromiso Técnico

Al firmar este documento, los miembros del equipo de ingeniería de **ACIDE** declaran:

1. **Comprensión Total:** He leído y comprendido las capas de la arquitectura, las reglas de dependencia y la diferencia entre una **Lib** técnica y un **Módulo** de negocio.
2. **Responsabilidad de Calidad:** Acepto que mi código en los proyectos

locales debe seguir el estándar para permitir la futura extracción de activos al **Garaje**.

3. **Compromiso de Auditoría:** Me comprometo a revisar activamente las propuestas de mejora de mis compañeros en los plazos establecidos (48h / 1 semana) para fortalecer nuestro patrimonio técnico.
4. **Aceptación de Gobernanza:** Entiendo que el incumplimiento de las **Invariantes (Punto 5)** es motivo de rechazo automático de mis contribuciones.

Registro de Firmas (Control de Versión del Manifiesto)

Fecha	Nombre del Desarrollador	Usuario Git / Firma	Rol
19/03/2026	Eleo Romario Huayhua Paco	@eleoromario	Líder Arquitecto